Props vs State in React

- Understanding core concepts of React

What are Props?

- • Props = Properties
- • Passed from parent to child
- • Read-only
- • Used for communication

Props Example

- function Welcome(props) {
- return <h1>Hello, {props.name}</h1>;
- }

- <Welcome name='Saba' />

What is State?

- • State = Internal data
- • Managed inside component
- • Can change over time
- • Controls dynamic behavior

State Example

- const [count, setCount] = useState(0);

- <button onClick={() => setCount(count+1)}>plus</button>
- <h1> counter </h1>
- <button onClick={() => setCount(count1)}>plus</button>

Props vs State (Difference)

- Props:
- • Read-only
- • From parent

- State:
- • Mutable
- • Inside component

Simple Analogy

- Props = Gift from parents (cannot change)
- State = Mood (can change anytime)

When to Use?

- Use Props → Pass data
- Use State → Dynamic UI changes

What are Hooks?

- • Hooks let you use state and features in functional components
- • Introduced in React 16.8
- • No need for class components

Common Hooks

- • useState → Manage state
- • useEffect → Side effects
- • useContext → Global data
- • useRef → Access DOM

useState Hook

- const [count, setCount] = useState(0)

- • count = state
- • setCount = update function

useEffect Hook

- useEffect(() => {
- console.log('Component Mounted');
- }, []);

- • Runs after render
- • Used for API calls, etc.

Why Hooks?

- • Simpler code
- • Reusable logic
- • No class components needed
- • Better readability

Lists in React

- • Used to display multiple items
- • Use JavaScript map() method
- • Each item must have a unique key

List Example

- const students = ['Ali','Sara','Saba'];

- students.map((name,index)=>(
- <li key={index}>{name}
-))

Keys in React

- • Keys help React identify elements
- • Should be unique
- • Avoid using index if possible

Conditional Rendering

- • Used to display UI based on conditions
- • Works like JavaScript conditions

If Condition Example

- if(isLoggedIn){
- return <h1>Welcome</h1>
- }else{
- return <h1>Please Login</h1>
- }

Ternary Operator

- isLoggedIn ? <h1>Welcome</h1> : <h1>Please Login</h1>

Logical AND (&&)

- isLoggedIn && <h1>Welcome</h1>

- • Shows only if condition is true